

REPASO - E. D.

El comando que prepara archivos para commit es: git add.

Las pruebas de aceptación buscan verificar algoritmos.

Los sprints en Scrum suelen durar: 1 día.

Límite WIP: evitar sobrecarga de trabajo

KANBAN: flujo continuo y visualización

Scrum: framework ágil

Daily scrum: 15 min

Product Owner: priorizar backlog (el total de tareas) según el valor para el cliente.

Sprints: 1 - 4 semanas

XP: calidad técnica

Patrón de diseño: solución reutilizable a un problema común

Command encapsula Acciones como objetos.

Patrones creacionales: creac. De objetos.

Patrón State : comportamiento depende del estado.

Strategy permite cambiar algoritmos en tiempo de ejecución

Observer: notificar cambios a observadores.

Decorator: añadir funcionalidades dinámicamente.

Adapter: traducir interfaces.

Factory: crear objetos sin exponer lógica

Singleton: una única instancia

Command encapsula acciones como objetos.

Pilares de Scrum: Transparencia, inspección, adaptación.

SOLID: principios de diseño.

La validación en testing comprueba: Si el producto satisface las necesidades del usuario.

Una técnica típica de caja blanca es Cobertura de sentencias.

diagrama UML se parece más a un diagrama de flujo Diagrama de actividad

Los diagramas de secuencia muestran: El orden temporal de mensajes entre objetos

Un diagrama de actividad UML se utiliza para representar: El flujo de tareas y decisiones

herramienta UML permite generar código y diagramas: StarUML

Git

Estrategias de Branching (cómo se organizan las ramas y cómo fluye el trabajo)

- **Git Flow:** múltiples ramas (main, develop, feature, release, hotfix). control exhaustivo, menos ágil
- **GitHub Flow:** Flujo sencillo basado en una rama main y ramas de características (feature/*). Ideal para integración continua (CI/CD), despliegues rápidos y aplicaciones web
- **Trunk Based Development:** Todos los desarrolladores trabajan directamente sobre main realizando commits diarios muy frecuentes. Requiere altísimo nivel de testing automatizado y madurez DevOps
- **Hotfix:** Ramas de emergencia creadas para corregir en producción de forma muy rápida

Comandos Clave en el Manejo de Ramas

- **Crear y cambiar a una nueva rama:** `git switch`
- **Fusionar una rama (desde main):** `git merge`
- **Eliminar una rama localmente:** `git branch`
- **Ver el historial gráfico de ramas:** `git log`

Scrum

Principios: transparencia, inspección, adaptación

Roles:

- **Product Owner (PO):** Representa al cliente. Decide qué se va a hacer.
- **Scrum Master (SM):** Es el guía del equipo.
- **Development Team (DT):** Se encargan de desarrollar, probar y entregar el producto funcional.

Artefactos

Representan el trabajo en el proyecto:

- **Product Backlog:** Es la lista priorizada de los requisitos e historias de usuario del producto.
- **Sprint Backlog:** Es el subconjunto de tareas del Product Backlog que se eligen para completarse en el Sprint actual.
- **Incremento:** Es el resultado real y funcional que se obtiene al terminar el Sprint.

Eventos

- **Sprint Planning:** Ocurre al inicio del Sprint y sirve para definir los objetivos.
- **Daily Scrum:** reunión diaria y breve para revisar qué se hizo el día anterior, qué se hará hoy y si existen problemas.
- **Sprint Review:** Se realiza al final del Sprint
- **Sprint Retrospective:** Ocurre justo después de la Review; el equipo analiza cómo fue su forma de trabajar y qué cosas pueden mejorar

Ventajas

- entregas de producto frecuentes y funcionales.
- comunicación y motivación.

- Flexibilidad.

Kanban

Consiste en organizar el trabajo mediante un un tablero visual y tarjetas.

Elementos clave:

- **Tablero:** Lugar (físico o digital) donde se visualizan las tareas y su estado.
 - o **Backlog:** historias de usuario.
 - o **To Do:** HU que trabajaremos ahora.
 - o **Doing:** tareas en las que estamos trabajando.
 - o **Review/Testing:** tareas que hay que revisar y probar.
 - o **Done:** tareas finalizadas.
- **Tarjetas:** Cada una representa una tarea individual.
- **Columnas:** Representan las diferentes fases del trabajo, habitualmente divididas en "Por hacer" (To Do), "En proceso" (Doing) y "Hecho" (Done).

Límite WIP (Work In Progress): Es la norma más importante de Kanban y consiste en limitar la cantidad de tareas En proceso para evitar hacer demasiadas cosas a la vez.

XP

Prácticas principales:

- **Programación en pareja:**
- **TDD (Test-Driven Development):** escribir los tests antes
- **Refactorización:**
- **Integración continua:** Varias veces al día.

Lean

Eliminación del Desperdicio (Waste) - EFICIENCIA

Patrones de diseño

CREACIONALES

Se centran en cómo se crean los objetos, separando la decisión de "cómo se crea" del objeto del código que lo va a utilizar.

Tipos

- **Singleton:** Una única instancia de un recurso mediante un método estático para acceder a ella. (ej: conexión a BDD)
- **Factory Method:** Delega la decisión de crear un objeto a un método protegido o abstracto.
- **Abstract Factory:** Sirve para crear familias-grupos de objetos que tienen que ir a juego entre sí (es como una fábrica de fábricas).
- **Builder:** Evitar constructores con exceso de parámetros separando la construcción de un objeto de su representación final.

ESTRUCTURALES

Se enfocan en cómo organizar y conectar las clases y objetos para que el sistema pueda crecer sin romperse.

- **Adapter:** Soluciona la colaboración entre clases e interfaces creando un adaptador-método que implementa el uso de otro método de una librería.
- **Composite:** Permite que objetos individuales y conjuntos de ellos sean tratados de la misma forma.
- **Decorator:** Permite añadir capacidades nuevas a un objeto de forma dinámica, manteniendo una interfaz común y evitando una jerarquía gigante de subclases.

En otras palabras: cambia la herencia (extends) por la **composición** (meter un objeto dentro de otro).

DE COMPORTAMIENTO:

- **Command:** es un patrón de diseño de comportamiento cuya misión principal es exactamente esa: transformar una solicitud, petición o acción en un objeto independiente que contiene toda la información necesaria para ejecutarse.

- **Observer (Observador):** Define una dependencia de "uno a muchos". Cuando el objeto principal (sujeto) cambia de estado, notifica automáticamente a todos los objetos dependientes (observadores).

Ejemplo típico: Es el corazón del patrón arquitectónico **MVC**. La Vista "observa" al Modelo para actualizarse cuando los datos cambian.

- **Strategy (Estrategia):** Permite que el algoritmo varíe independientemente del cliente que lo utiliza.

Ejemplo típico: Un sistema de pagos en un e-commerce. Tienes una interfaz `EstrategiaPago` y clases que la implementan como `PagoPayPal`, `PagoTarjeta`.

- **State (Estado):** Permite a un objeto alterar su comportamiento cuando su estado interno cambia. Parecerá que el objeto ha cambiado de clase.

Ejemplo típico: El ciclo de vida de un pedido en una tienda online

- **Iterator (Iterador):** Proporciona una forma de acceder secuencialmente a los elementos de una colección

Ejemplo típico: Los bucles `for-each` y la interfaz `Iterator` de las colecciones en Java.

SOLID

Relación: Los principios SOLID existen fundamentalmente para aumentar la cohesión (donde cada clase tiene un objetivo único claro) y reducir el acoplamiento (que un cambio en una parte del código no afecte al resto).

S - Single Responsibility Principle

O - Open/Closed Principle (Abierto / Cerrado)

El software debe estar abierto a extensión, pero cerrado a modificación (no tocar el código estable).

L - Liskov Substitution Principle (Sustitución de Liskov)

Una clase hija debe poder sustituir a su clase padre sin que el funcionamiento del programa se vea alterado.

I - Interface Segregation Principle (Segregación de Interfaces)

Es mejor tener muchas interfaces pequeñas y específicas que tener una sola interfaz gigante y genérica. Ninguna clase debería implementar métodos que no utiliza.

D - Dependency Inversion Principle (Inversión de Dependencias)

Evita depender de tecnologías concretas (como MySQL directamente

Code smells

CATÁLOGO COMÚN DE CODE SMELLS

- **Long Method (Métodos largos)**
- **God Class (Clases Dios)**
- **Duplicated Code (Código duplicado)**
- **Large Parameter List**
- **Feature Envy:** Ocorre cuando una clase utiliza más datos de otra clase que los suyos propios.
- **Primitive Obsession:** Abuso de tipos primitivos.
- **Switch/If complejos**
- **Nombres pobres**

Refactorización

TÉCNICAS BÁSICAS DE REFACTORIZACIÓN

- **Rename:** Consiste en cambiar el nombre de las variables, métodos o clases.
- **Extract Method:** Se basa en tomar fragmentos de código complejos y extraerlos a métodos independientes y más pequeños
- **Extract Variable:** Implica guardar expresiones que son largas o complejas dentro de variables con un nombre descriptivo para facilitar su lectura.
- **Extract Class:** Consiste en separar las responsabilidades creando nuevas clases.

Pruebas

- **Unitarias (Nivel código):** ¿Funciona esta pieza en concreto?
- **De integración (Nivel módulos):** ¿Encajan bien las diferentes piezas?
- **Pruebas de sistema (Nivel sistema):** ¿Funciona el todo en su conjunto?
- **Pruebas de aceptación (Nivel usuario):** ¿Sirve para quien lo va a usar?

El tester no arregla el software, sino que se encarga de evaluar los riesgos.

CI/CD (Integración y Despliegue Continuo):

- **CI:** Cada vez que el desarrollador sube código, el sistema compila y ejecuta automáticamente las pruebas.
- **CD:** Si todas las pruebas pasan correctamente, el sistema puede desplegarse automáticamente.

Técnicas de Caja Negra

- **Partición de Equivalencia:** si un valor de un grupo funciona bien (o da el error esperado), todos los demás valores de ese mismo grupo harán lo mismo.
- **Análisis de Valores Límite:** Consiste en probar los límites exactos de una restricción (ej. si el mínimo son 8 caracteres, probar con 7, 8 y 9), ya que ahí suelen ocurrir más errores.
- **Casos de Uso:** Se prueban escenarios y flujos reales de usuario (ej. iniciar sesión, añadir producto y comprar).

Técnicas de Caja Blanca

- **Cobertura de Sentencias:** Asegura que cada línea de código se ha ejecutado
- **Cobertura de Decisiones:** Asegura que cada condición ha sido evaluada tanto en su estado true como en false.
- **Cobertura de Caminos:** Consiste en probar todas las rutas lógicas posibles.

PRUEBAS NO FUNCIONALES

No responden a qué hace el sistema, sino a *cómo lo hace*, evaluando atributos globales de calidad

- **Rendimiento:** tiempo de respuesta, capacidad bajo carga y consumo de recursos.
- **Seguridad:** Detectan vulnerabilidades.
- **Usabilidad y Compatibilidad.**

Anotaciones de Ciclo de Vida (Testing)

- **@Test:** Indica que ese método es una prueba a ejecutar.
- **@BeforeEach / @AfterEach:** Se ejecutan automáticamente justo antes y justo después de **cada** prueba individual (ideal para preparar configuraciones como inicializar objetos).
- **@BeforeAll / @AfterAll:** Se ejecutan una única vez al principio o al final de **todos** los tests de la clase (deben ser estáticos).

Métricas de calidad

COMPLEJIDAD CICLOMÁTICA (CC)

Mide el número de caminos que puede seguir el flujo de ejecución del código.

Fórmula Teórica resumida: n° de decisiones + 1

Cuentan como una decisión: Sentencias if, else if, for, while, case, catch y operadores lógicos como && o ||.

MÉTRICA CRAP

Mide **el riesgo y el esfuerzo necesario para modificar un trozo de código** basándose en la relación entre la complejidad y su porcentaje de cobertura de pruebas (coverage).

TDD, ATDD y BDD aplicados al desarrollo web ... “N/A”

TDD (TEST-DRIVEN DEVELOPMENT)

Es una metodología donde se escribe la prueba unitaria antes de escribir el código funcional. **Ciclo de TDD (Red - Green - Refactor):**

- **RED (Rojo):** Se escribe una prueba que va a fallar (porque la funcionalidad aún no existe).
- **GREEN (Verde):** Se escribe el código mínimo e indispensable para conseguir que la prueba pase con éxito.
- **REFACTOR (Refactorizar):** Se mejora y limpia el código manteniendo la prueba en verde (asegurando que no se rompe nada).

ATDD (Acceptance Test-Driven Development)

Este enfoque se basa en que las pruebas se definen a nivel funcional directamente junto con el cliente antes de empezar a programar (ej. qué pasa si hay saldo o si no hay saldo en una transferencia).

BDD (Behavior-Driven Development)

Utiliza lenguaje natural estructurado para describir el comportamiento del sistema con el objetivo de mejorar la comunicación entre los desarrolladores, QA (Testing) y el cliente. **GWT:**

- o **Given (Dado):** Un contexto inicial.
- o **When (Cuando):** Ocurre una acción.
- o **Then (Entonces):** Se espera un resultado.

Diagramas

DIAGRAMA DE CASOS DE USO (VISIÓN EXTERNA) *¿Qué puede hacer el usuario con el sistema?*

- **Actor:** Quien usa el sistema.
- **Caso de uso:** La acción que realiza.
- **Sistema:** El límite de la aplicación.

DIAGRAMA DE SECUENCIA (INTERACCIÓN INTERNA) *¿Qué mensajes se envían los objetos.* Representa el orden temporal mediante "líneas de vida" (lifelines), mensajes y llamadas a métodos con sus respectivas respuestas.

DIAGRAMA DE Actividad (FLUJO LÓGICO) *¿Qué pasos sigue un proceso y qué decisiones o bifurcaciones aparecen en el camino?*

Se parece mucho a un diagrama de flujo tradicional

DIAGRAMA DE ESTADOS (CICLO DE VIDA) *¿Por qué fases o estados pasa un objeto concreto a lo largo de su vida?*

se centra en la evolución de un único objeto o entidad